

27 January 2026

Reverse Engineering and Experimental Analysis of a Deterministic Enemy AI in Pac-Man

Chhanon Teng

Abstract

Early arcade video games relied on deterministic, rule-based artificial intelligence due to extreme hardware and memory constraints. Despite their simplicity, these systems produced complex and engaging enemy behaviors that continue to influence modern game AI design. This paper presents a reverse engineering and experimental analysis of Blinky, the red ghost in Pac-Man. Using a modified open-source implementation with step-controlled execution, Blinky's movement behavior was isolated and analyzed at each decision point. Experimental results show that Blinky follows a greedy Manhattan-distanceminimization strategy during its chase mode, while operating under a distinct roaming behavior prior to chase activation. The study demonstrates how simple deterministic rules, behavioral modes, and maze constraints interact to produce emergent pursuit behavior.

Keywords

deterministic algorithms, game ai, greedy search, pac-man, photonics and electrooptics, reverse engineering

Reverse Engineering and Experimental Analysis of a Deterministic Enemy AI in Pac-Man

CHHANON TENG

Department of Computer Science
Paragon International University
Email: cteng@paragoniu.edu.kh

Abstract—Early arcade video games relied on deterministic, rule-based artificial intelligence due to extreme hardware and memory constraints. Despite their simplicity, these systems produced complex and engaging enemy behaviors that continue to influence modern game AI design. This paper presents a reverse engineering and experimental analysis of Blinky, the red ghost in *Pac-Man*. Using a modified open-source implementation with step-controlled execution, Blinky’s movement behavior was isolated and analyzed at each decision point. Experimental results show that Blinky follows a greedy Manhattan-distance-minimization strategy during its chase mode, while operating under a distinct roaming behavior prior to chase activation. The study demonstrates how simple deterministic rules, behavioral modes, and maze constraints interact to produce emergent pursuit behavior.

Index Terms—Game AI, Pac-Man, Reverse Engineering, Deterministic Algorithms, Greedy Search

I. INTRODUCTION

Artificial intelligence in early video games was constrained by limited processing power, minimal memory, and strict real-time execution requirements. As a result, developers relied on deterministic and handcrafted rule-based systems rather than probabilistic or learning-based approaches [1], [2]. These constraints forced designers to produce algorithms that were computationally inexpensive yet capable of creating engaging and challenging gameplay.

One of the most influential examples of early game AI is *Pac-Man*, released by Namco in 1980 [3]. The game features four enemy characters, commonly referred to as ghosts, whose behaviors were intentionally differentiated to control difficulty and player experience. Rather than relying on randomness, each ghost was designed to follow a specific movement logic that contributed to the game’s strategic depth [4], [8].

Among the four ghosts, Blinky (the red ghost) is characterized by its aggressive pursuit behavior. Unlike other ghosts that incorporate predictive targeting or semi-random movement, Blinky directly attempts to reduce its distance to Pac-Man, making it an ideal subject for deterministic algorithm analysis [9], [11]. Although *Pac-Man* ghost behavior has been widely explained in popular media and developer documentation, fewer studies provide experimentally controlled validation of these behaviors.

The objective of this paper is to reverse engineer Blinky’s movement behavior and validate its chase algorithm through controlled experimentation. The study focuses on identifying

the decision rule governing Blinky’s movement, examining the effect of maze constraints, and analyzing the role of behavioral modes in shaping observed behavior.

II. METHODOLOGY

A. Experimental Setup

Experiments were conducted using a modified open-source Python implementation of *Pac-Man* built on the Pygame framework. The codebase was modified by the author to isolate Blinky as the only active ghost while fixing Pac-Man at a constant position to eliminate player-induced variability.

A step-controlled execution mode was implemented, allowing Blinky to advance exactly one maze node per user input. Each step corresponds to a single movement decision made at a maze junction. At every step, Blinky’s position, Pac-Man’s position, and the selected movement direction were logged. This controlled setup enables reproducible inspection of Blinky’s decision-making process.

B. Movement Logging and Coordinate Collection

To capture Blinky’s behavior comprehensively, movement logging was initiated at the ghost’s spawn position and continued uninterrupted until Pac-Man was reached. This approach allows observation of both the initial roaming behavior and the later chase behavior within a single continuous sequence.

Table I summarizes a representative subset of recorded movement steps. Pac-Man’s position was held constant at (216, 416) for all experiments to ensure that observed behavior resulted solely from Blinky’s decision logic rather than player movement.

TABLE I
SAMPLE LOGGED BLINKY MOVEMENT COORDINATES

Step	Blinky (x, y)	Pac-Man (x, y)	Direction
1	(216, 224)	(216, 416)	Down
2	(192, 224)	(216, 416)	Left
3	(192, 176)	(216, 416)	Down
4	(144, 176)	(216, 416)	Left
5	(144, 128)	(216, 416)	Down
6	(96, 128)	(216, 416)	Left
7	(96, 64)	(216, 416)	Down
8	(16, 64)	(216, 416)	Left
9	(16, 128)	(216, 416)	Up

Early movement steps do not consistently reduce the distance between Blinky and Pac-Man. Instead, Blinky traverses

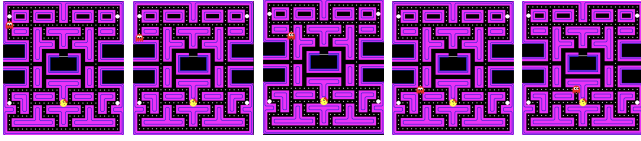


Fig. 1. Representative screenshots illustrating Blinky's chase behavior after the roaming-to-chase transition.

a bounded region of the maze, indicating operation under a non-chasing roaming mode.

C. Roaming-to-Chase Mode Transition

A clear behavioral transition was observed when Blinky reached approximately position (16, 128). Beyond this node, Blinky's movement pattern changed qualitatively, exhibiting consistent distance-reducing behavior toward Pac-Man.

Importantly, algorithmic testing of Blinky's chase logic was conducted only after this transition point. Movement decisions made during the roaming phase were excluded from chase analysis to avoid conflating scripted or mode-based behavior with the pursuit algorithm itself.

D. Chase Decision Model

During chase mode, Blinky's movement decisions were observed to follow a greedy distance-minimization strategy. At each maze node, Blinky evaluates all valid movement directions except the immediate reverse of its current direction.

For each candidate direction d , the Manhattan distance to Pac-Man is computed as:

$$D(d) = |x_g(d) - x_p| + |y_g(d) - y_p| \quad (1)$$

where $(x_g(d), y_g(d))$ represents Blinky's potential next position if direction d is chosen, and (x_p, y_p) represents Pac-Man's fixed position.

The selected direction is therefore:

$$d^* = \arg \min_{d \in \mathcal{D}} D(d) \quad (2)$$

where $\mathcal{D}$ denotes the set of valid movement directions at the current node. This formulation confirms that Blinky performs a locally optimal greedy search rather than global path planning.

E. Visualization of Chase-Phase Movement

To visually illustrate Blinky's behavior during chase mode, representative screenshots were captured at key decision points following the roaming-to-chase transition. These images correspond to consecutive movement decisions beginning at chase activation and ending near Pac-Man's position.

Figure 1 presents five representative frames showing the progression of Blinky's pursuit. Only a subset of screenshots is included to maintain clarity and adhere to space constraints. The full movement log and complete screenshot set are provided in the Appendix.

III. RESULTS

Analysis of Blinky's movement during the chase phase reveals a deterministic pursuit strategy. At each decision point, Blinky selects the direction that minimizes the Manhattan distance to Pac-Man, subject to maze connectivity and movement constraints that prohibit immediate reversal.

Across all observed chase-phase movements, Blinky consistently reduced its distance to Pac-Man whenever a valid distance-reducing path was available. Temporary increases in distance occurred only when forced by maze geometry, such as corridors or dead ends. Once an alternative path became available, Blinky immediately resumed distance minimization.

A. Effect of Maze Constraints

Maze geometry plays a critical role in shaping Blinky's observed path. Although the underlying decision rule is purely local, the maze layout constrains available choices, producing movement patterns that may appear strategic despite the absence of prediction or planning. This interaction between simple algorithms and environmental structure is a key source of emergent behavior in classic game AI systems [5].

IV. DISCUSSION

The experimental findings confirm that Blinky's chase behavior is governed by a greedy distance-minimization algorithm. This aligns with broader studies of deterministic enemy AI in arcade games and early interactive systems [1], [4]. Despite lacking prediction or global planning, Blinky remains effective due to frequent decision points and the constrained topology of the maze.

The observed roaming-to-chase transition highlights the importance of behavioral modes in early game AI design. Similar mode-based control architectures have been documented in academic studies of Pac-Man and competitive implementations [6], [7]. By separating scripted roaming from active pursuit, designers can regulate difficulty pacing while maintaining deterministic logic.

From an educational perspective, Pac-Man provides a valuable case study for teaching greedy algorithms, state-based control, and emergent behavior. The transparency and reproducibility of its AI logic make it particularly suitable for reverse engineering and experimental analysis.

V. CONCLUSION

This paper presented a reverse engineering and experimental analysis of Blinky's movement behavior in *Pac-Man*. Through controlled step-by-step execution, Blinky was shown to follow a deterministic greedy pursuit algorithm during its chase mode, minimizing Manhattan distance to Pac-Man while respecting maze constraints.

The study demonstrates how simple deterministic rules combined with behavioral modes and environmental design can generate effective and engaging enemy behavior. The methodology presented provides a reproducible framework for analyzing classic game AI systems and may be extended to other ghosts or comparative studies with modern AI techniques.

APPENDIX A

USE OF ARTIFICIAL INTELLIGENCE TOOLS

Artificial intelligence tools, including large language models, were used solely to assist with language refinement, structural organization, and LaTeX formatting. All experimental design, code modification, data collection, analysis, interpretation, and conclusions were performed entirely by the author.

REFERENCES

- [1] I. Millington and J. Funge, *Artificial Intelligence for Games*, 2nd ed. CRC Press, 2009.
- [2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.
- [3] Namco, *Pac-Man*, Arcade Game, 1980.
- [4] G. N. Yannakakis and J. Togelius, "Player Modeling and Adaptation in Games," *Artificial Intelligence*, vol. 174, no. 14, pp. 1056–1079, 2010.
- [5] G. N. Yannakakis and J. Togelius, "A Panorama of Artificial and Computational Intelligence in Games," *IEEE Trans. Computational Intelligence and AI in Games*, 2014.
- [6] S. M. Lucas, "Ms Pac-Man Competition," *IEEE Symposium on Computational Intelligence and Games*, 2007.
- [7] S. M. Lucas, "Evolving a Neural Network Location Evaluator to Play Ms. Pac-Man," *IEEE Symposium on Computational Intelligence and Games*, 2005.
- [8] The Pac-Man Dossier, Gamasutra. https://www.gamasutra.com/view/feature/3938/the_pacman_dossier.php
- [9] Pac-Man Strategy Wiki. <https://strategywiki.org/wiki/Pac-Man>
- [10] Pac-Man Strategy and Ghost Behavior, ClassicGaming.cc. <https://www.classicgaming.cc/classics/pac-man/strategy>
- [11] "Pac-Man Ghost AI Explained," YouTube, 2019. <https://youtu.be/ataGotQ7ir8>
- [12] A. L. Samuel, "Some Studies in Machine Learning Using the Game of Checkers," *IBM Journal of Research and Development*, 1959.